1385. Find the Distance Value Between Two Arrays

Daily Question - November 5, 2026

Problem Statement

Given two integer arrays arr1 and arr2, and the integer d, return the distance value between the two arrays.

The distance value is defined as the number of elements arr1[i] such that there is not any element arr2[j] where |arr1[i] - arr2[j]| <= d.

Examples

Example 1: arr1 = [4,5,8], arr2 = [10,9,1,8], d = 2. Output: 2. Values 4 and 5 have no arr2 value within distance 2; value 8 is within distance 2 of 8 and 9.

Example 2: arr1 = [1,4,2,3], arr2 = [-4,-3,6,10,20,30], d = 3. Output: 2.

Example 3: arr1 = [2,1,100,3], arr2 = [-5,-2,10,-3,7], d = 6. Output: 1.

Constraints

1 <= arr1.length, arr2.length <= 500; -1000 <= arr1[i], arr2[j] <= 1000; 0 <= d <= 100.

Solution: Sorting + Binary Search

Sort arr2. For each value x in arr1, binary-search for the first index position such that arr2[position] >= x. The only arr2 values that can possibly be within distance d of x are arr2[position - 1] and arr2[position], the closest values on either side.

Count x if both neighbors are absent or farther than d: value - arr2[position - 1] > d and arr2[position] - value > d. This means no element of arr2 satisfies |x - arr2[j]| <= d.

Complete Java Solution

```
import java.util.Arrays;

class Solution {
    public int findTheDistanceValue(int[] arr1, int[] arr2, int d) {
        Arrays.sort(arr2);

        int count = 0;

        for (int value : arr1) {
            int position = lowerBound(arr2, value);

            boolean leftFar =
                    position == 0
                    || value - arr2[position - 1] > d;

            boolean rightFar =
                    position == arr2.length
```

```
                    || arr2[position] - value > d;

            if (leftFar && rightFar) {
                count++;
            }
        }

        return count;
    }

    private int lowerBound(int[] arr, int target) {
        int left = 0;
        int right = arr.length;

        while (left < right) {
            int middle = left + (right - left) / 2;

            if (arr[middle] < target) {
                left = middle + 1;
            } else {
                right = middle;
            }
        }

        return left;
    }
}
```

Correctness

After sorting arr2, the closest element below x is arr2[position - 1] and the closest element at or above x is arr2[position]. If either is within distance d, then x does not contribute to the answer. If both are absent or farther than d, every other element of sorted arr2 is even farther away, so x contributes exactly one to the count. Summing these contributions gives the distance value.

Complexity

Let m = arr1.length and n = arr2.length. Time is O(n log n + m log n): sorting arr2 takes O(n log n), and each of the m binary searches takes O(log n). Auxiliary space is O(1), ignoring the sorting stack.

Implementation Notes

The lowerBound method returns the first index whose value is at least the target. Handle the boundary cases where position is 0 or arr2.length; in those cases, only the existing neighbor needs checking. A simple nested-loop solution is also valid under the small constraints, but this approach is more scalable.